StockPulse — Python VaR Backend: Architecture & Setup Guide

Overview

This is the **server-side Python pipeline** that powers the VaR calculations for the StockPulse iOS app. Instead of computing VaR on-device in Swift when a user opens a stock, this backend **pre-computes VaR daily** for every combination of confidence level, lookback window, and ticker — then writes the results to Supabase where the iOS app reads them instantly.

Architecture Diagram

```
┌──────────────────────────────────────────────────────────────────────────────────────────────┐
│                          DAILY PIPELINE (Python)                                             │
│                        Runs at 21:30 UTC (Mon–Fri)                                           │
│                                                │                                             │
│  ┌──────────────┐   ┌──────────────────┐   ┌──────────────────────────────┐   │
│  │  Step 1   │   │  Step 2     │   │      Step 3         │   │
│  │ Refresh   │──▶│ Backfill new │──▶│ Fetch today's close  │   │
│  │ S&P 500   │   │ tickers     │   │ for all 500 tickers  │   │
│  │ (Wikipedia)│   │ (Polygon)   │   │ (Polygon)           │   │
│  └──────────────┘   └──────────────────┘   └──────────────┬───────────────┘   │
│                                  │            │                                          │
│                                  ┌──────────────▼──────────────┐   │
│                                  │     Step 4         │   │
│                                  │ Compute VaR        │   │
│                                  │ 500 tickers ×      │   │
│                                  │ 200 confidence lvls ×  │   │
│                                  │ 5 lookback windows    │   │
│                                  │ = 500,000 rows/day    │   │
│                                  └──────────┬──────────────┘   │
│                                          │            │                                          │
│                                  ┌──────────────▼──────────────┐   │
│                                  │     Step 5         │   │
│                                  │ Upsert to Supabase    │   │
│                                  │                 │   │
│                                  └──────────────────────────┘   │
└──────────────────────────────────────────────────────────────────────────────────────────────┘
                                  │
                                  ▼
┌──────────────────────────────────────────────────────────────────────────────────────────────┐
│                          SUPABASE (PostgreSQL)                                               │
│                                                │                                             │
│  ┌──────────────────────────┐  ┌──────────────────────────────────────────────┐   │
│  │  sp500_constituents  │  │  var_calculations_precomputed  │   │
```

```
│  │ (500 rows)        │  │ (~500,000 rows/day, kept all) │   │
│  └──────────────────────┘  └──────────────────────────────────┘   │
│  ┌──────────────────────┐  ┌──────────────────────────────────┐   │
│  │ price_history       │  │ watchlist_stocks (per-user)   │   │
│  │ (~630,000 rows)    │  │                              │   │
│  └──────────────────────┘  └──────────────────────────────────┘   │
│                                        │
│ RPC functions:                                  │
│ • get_var_for_stock(ticker, confidence, lookback)          │
│ • get_var_all_levels(ticker, lookback)  ← for slider UI     │
└──────────────────────────────────────────────────────────────────────┘
                    │
                    ▼
┌──────────────────────────────────────────────────────────────────────┐
│                iOS APP (Swift/SwiftUI)                  │
│                                        │
│ User adds AAPL → app calls get_var_for_stock("AAPL", 0.95, 252)│
│ User drags slider → app calls get_var_all_levels("AAPL", 252) │
│ Results are INSTANT — no on-device calculation needed          │
└──────────────────────────────────────────────────────────────────────┘
```

File Structure

```
stockpulse_backend/
├── main.py                  ← CLI entry point
├── scheduler.py             ← APScheduler daemon (alternative to cron)
├── requirements.txt         ← Python dependencies
├── .env.example             ← Environment variable template
│
├── config/
│   └── settings.py          ← All configuration (Supabase, Polygon, VaR params)
│
├── services/
│   ├── supabase_client.py   ← Supabase read/write operations
│   ├── polygon_client.py    ← Polygon.io OHLCV data fetching
│   ├── sp500_tracker.py     ← S&P 500 list refresh (Wikipedia scrape)
│   └── var_engine.py        ← Core VaR math (Historical Simulation)
│
├── jobs/
│   ├── daily_pipeline.py    ← Main orchestration (Steps 1–5)
│   └── backfill.py          ← Historical data loader for new tickers
│
```

```
└── migrations/
    └── 002_var_precomputed_and_sp500.sql  ← New Supabase tables & RPC functions
```

What Gets Computed

Dimension	Values	Count
Tickers	All active S&P 500 constituents	~503
Confidence levels	80.0%, 80.1%, 80.2%, ..., 99.8%, 99.9%	200
Lookback windows	252d (1yr), 504d (2yr), 756d (3yr), 1008d (4yr), 1260d (5yr)	5
Total rows per day	503 × 200 × 5	**~503,000**

The VaR method is **Historical Simulation** (matches your design doc's default). Each row stores both the percentage return boundaries and the absolute price boundaries.

Setup Steps

1. Prerequisites

- Python 3.11+
- A Supabase project (free tier works)
- A Polygon.io API key (free tier: 5 calls/min)

2. Install Dependencies

```bash
cd stockpulse_backend
pip install -r requirements.txt
```

3. Configure Environment

```bash
cp .env.example .env
# Edit .env with your actual keys:
#   SUPABASE_URL, SUPABASE_SERVICE_ROLE_KEY, POLYGON_API_KEY
```

> **Important:** Use the Supabase **Service Role** key (not the anon key). This backend needs to bypass Row Level Security to write to shared tables. Find it in Supabase → Settings → API → `service_role` (the secret one).

4. Run the SQL Migration

Go to your Supabase dashboard → **SQL Editor** → paste and run the contents of:

```
migrations/002_var_precomputed_and_sp500.sql
```

This creates:

- `sp500_constituents` table
- `var_calculations_precomputed` table (with indexes)
- `get_var_for_stock()` RPC function
- `get_var_all_levels()` RPC function
- RLS policies (public read, backend-only write)

5. Initial Run

```bash
# Step A: Populate the S&P 500 list
python main.py --refresh-sp500

# Step B: Full historical backfill (takes hours on free tier)
python main.py --initial-backfill

# Or backfill a single ticker to test
python main.py --backfill AAPL
```

6. Schedule Daily Runs

Option A — Cron (recommended for VPS/server):

```bash
# Run at 21:30 UTC (Mon–Fri), 30 min after US market close
30 21 * * 1-5 cd /path/to/stockpulse_backend && python main.py --run-daily >> /var/log/stockpulse.log 2>&1
```

Option B — APScheduler daemon:

```bash
python scheduler.py
# Runs as a long-lived process, triggers at 21:30 UTC Mon–Fri
```

Option C — GitHub Actions (free, no server needed):

```yaml
# .github/workflows/daily-var.yml
name: Daily VaR Pipeline
on:
  schedule:
    - cron: '30 21 * * 1-5'   # 21:30 UTC Mon–Fri
  workflow_dispatch:          # manual trigger

jobs:
  run:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: '3.12' }
      - run: pip install -r stockpulse_backend/requirements.txt
      - run: python stockpulse_backend/main.py --run-daily
        env:
          SUPABASE_URL: ${{ secrets.SUPABASE_URL }}
          SUPABASE_SERVICE_ROLE_KEY: ${{ secrets.SUPABASE_SERVICE_ROLE_KEY }}
          POLYGON_API_KEY: ${{ secrets.POLYGON_API_KEY }}
```

How the iOS App Consumes This

The pre-computed VaR table replaces on-device calculation. In your Swift code, instead of running the VaR engine locally, you query Supabase:

When user opens a stock (specific confidence + lookback):

```swift
// Calls the get_var_for_stock Postgres function
let result = try await supabase.rpc("get_var_for_stock", params: [
    "p_ticker": "AAPL",
    "p_confidence": 0.950,
    "p_lookback": 252
]).execute()
```

When user drags the confidence slider (all levels for one lookback):

```swift
```

```
// Returns all 200 rows (80.0%–99.9%) for instant slider response
let result = try await supabase.rpc("get_var_all_levels", params: [
    "p_ticker": "AAPL",
    "p_lookback": 252
]).execute()
```

This means the slider in the Stock Detail screen is **instant** — all 200 confidence levels are already computed and cached. No waiting for on-device math.

Integrating Your Existing VaR Model

The file `services/var_engine.py` contains a clean `VaREngine` class. To swap in your existing Python VaR model:

1. Open `var_engine.py`
2. Replace the logic inside `compute_var_for_ticker()` with your own
3. Keep the **method signature and return format** the same — the rest of the pipeline depends on it

The key contract is:

- **Input:** ticker name, list of price dicts `[{date, close, adj_close}]`, calc_date, confidence levels, lookback windows
- **Output:** list of dicts matching the `var_calculations_precomputed` table schema

Data Flow for S&P 500 Changes

When a stock is **added** to the S&P 500:

1. `sp500_tracker.refresh()` detects the new ticker via Wikipedia diff
2. `backfill.backfill_ticker()` fetches 5 years of historical prices from Polygon
3. VaR is computed for today and written to the precomputed table
4. From tomorrow onward, the daily pipeline includes this ticker automatically

When a stock is **removed** from the S&P 500:

1. `sp500_tracker.refresh()` marks it as `is_active = FALSE`
2. The daily pipeline skips inactive tickers
3. Historical data and existing VaR rows are preserved (users may still have it on their watchlist)

Rate Limit Considerations

API	Free Tier Limit	Impact
Polygon.io	5 calls/min	Full backfill of 500 tickers ≈ 17 hours. Daily update of 500 tickers ≈ 17 hours. Upgrade to Basic ($29/mo) for unlimited.
Supabase	500 MB DB	~500K VaR rows/day, kept indefinitely. You'll need Supabase Pro ($25/mo) within the first month at production scale.

Recommendation for production: Get a Polygon paid plan. The free tier works for development and testing a handful of tickers, but processing all 500 daily is impractical at 5 calls/min. With a paid plan, set `polygon_rate_limit_per_min` higher in `settings.py` and reduce `price_fetch_delay_sec` accordingly.

Storage Estimation

Table	Rows	Est. Size
`price_history`	500 tickers × 1,260 days = 630,000	~80 MB
`var_calculations_precomputed`	500,000/day × 252 trading days/year	~15 GB/year
`sp500_constituents`	~503	negligible

VaR data is **kept indefinitely** (no pruning) so premium users can view historical VaR trends over months and years. The VaR table is the largest and will grow continuously. Options to manage growth:

- Reduce confidence granularity (every 0.5% instead of 0.1% → 40 levels instead of 200)
- Only compute for tickers that are on someone's watchlist (check `watchlist_stocks` first)
- Archive older data to cold storage periodically if Supabase storage limits are reached

---

Optimisation: Watchlist-Only Mode

If you want to reduce compute and storage, add this to the daily pipeline:

```python
```

```
# In daily_pipeline.py, replace all_tickers with:
watched = set(self.supabase.get_all_watched_tickers())
sp500 = set(self.sp500.get_active_tickers())
# Only compute VaR for stocks someone is actually watching
tickers_to_compute = list(watched & sp500)
```

This way you only compute VaR for the ~50–100 stocks that users have on their watchlists, not all 500. You can always fall back to on-device calculation for stocks not yet precomputed.